Diseño — E3: Mensajería, IA y guardarraíles (M6, M7, M18)

- **Fecha:** 2026-10-15
- **Estado:** propuesta para revisión de Leonardo
- **Spec fuente:** ALK-SRD-001 v2.0 §4.6 (M6), §4.7 (M7), §4.18 (M18), §3.2 (cuotas por plan) + Plan Maestro §4 E3 (`docs/superpowers/plans/2026-10-12-aicorelink-web-master-build-plan.md`)
- **Reemplaza a:** —
- **Depende de:** E2 (fábrica, completa), F0/F1 (panel, en prod), D10 del diseño del panel (`docs/superpowers/specs/2026-10-14-aicorelink-panel-architecture-design.md`)

---

1. Intención

Hoy el único tenant de producción no puede atender a nadie: `channel_credentials = 0` , `conversations = 0` , `messages = 0` . El panel muestra ceros honestos porque **no existe la ingesta**. La landing tiene el botón de WhatsApp y el widget de chat, pero del otro lado no hay nadie.

E3 es la etapa que cumple la promesa central del producto: **que cada sitio atienda de verdad**, con la calidad y los límites que el cliente compró. Su criterio de salida (Plan Maestro §9) es: *un tenant nuevo se activa cambiando sólo parámetros; calidad del chat ≥ 90%; pruebas adversarias de IA en verde*.

---

2. Alcance

2.1 Dentro de E3

Requisito	Prioridad	Qué exige	Etapa
RF-MSG-01	Alta	Canal WhatsApp sobre Meta Cloud API directa en producción; Evolution API sólo en desarrollo	E3.1
RF-MSG-02	Alta	Bandeja unificada (web, WhatsApp, voz) visible para el operador y acotada para el cliente	E3.1
RF-MSG-03	Alta	Derivación a humano con resumen; el dueño recibe aviso y la IA deja de responder en ese hilo	E3.4
RF-MSG-04	Alta	Flujos estándar en n8n versionados y clonables por cliente (lead, respuesta, reserva, recordatorio, reseña, reporte)	E3.5
RF-KB-01	Alta	Base de conocimiento propia por tenant (servicios, precios, horarios, políticas, FAQ), editable por operador y cliente	E3.2
RF-KB-02	Alta	La IA no puede inventar: ≥ 95% de 50 preguntas fuera de base termina en «no tengo ese dato» + derivación	E3.2
RF-KB-03	Alta	Temas prohibidos configurables; pruebas adversarias no logran que la IA los incumpla	E3.2
RF-KB-04	Media	Tono de marca configurable por cliente, con español de Chile	E3.2
RF-KB-05	Alta	Protección contra inyección de instrucciones: el contenido del cliente final nunca modifica reglas del sistema	E3.2
RF-KB-06	Media	Registro de conversaciones con evaluación periódica de calidad por muestreo	E3.6
RF-KB-07	Media	Selección de modelo por plan (local para simple, API directa para complejo o voz), política configurable y costo por conversación registrado	E3.3
RF-QLT-01	Alta	Modo aprendizaje: ante una consulta sin respuesta, el asistente pregunta al dueño y guarda la respuesta aprobada	E3.4
RF-QLT-04	Media	Medición del tiempo a primera respuesta, con datos reales por tenant	E3.6
RF-QLT-06	Media	Modelo y versión publicados + registro de cambios con prueba previa	E3.6
SRD §3.2 / RC-04	Alta	Cuotas del plan (conversaciones IA, cambios, plantillas, voz) con aviso al 80% y 100%; al agotarse, **el asistente nunca se apaga en silencio**	E3.3

2.2 Fuera de E3 (con su dónde)

Fuera	Por qué	Dónde vive
RF-MSG-05 plantillas de WhatsApp (alta/estado/uso)	Necesita la cuenta de Meta y su ciclo de aprobación	Con RF-MSX-04 (llamada perdida → plantilla), E7

Fuera	Por qué	Dónde vive
RF-MSG-06 reintentos y cola de mensajes fallidos	Requiere la plataforma de Chatwoot ya en pie y volumen real	E7
RF-MSG-07 numeración compartida (Starter)	**Decisión de negocio pendiente**; entrelazada con la coexistencia (RF-MSX-01)	E7 (ver §6, decisión de Leonardo)
RF-MSX-01..11 (coexistencia, notas de voz, Instagram, llamada perdida, datos y orden, respuestas fijas, estados y filtros, cuotas por canal, chat web unificado, seguimiento 24/72 h, bolsas de avisos)	Etapa propia; la mayoría es E7/E8	E7/E8
Recepcionista de voz (Elite, ≤ 1,5 s por turno)	Infra y latencia propias; `lib/voice` ya existe	E3.7 (ver §6) o E7
RF-QLT-02 panel de transparencia	Es UI del panel	F4 del panel
RF-QLT-03 revisión humana de los primeros 30 días	Proceso humano, no código	Operación (E5)
RF-QLT-05 gate automático de evaluación en CI	La suite se construye en E3.6; el gate corre cuando exista CI de despliegue	E7
RF-QLT-07 centro de confianza público	Marketing	E6
Facturación / cobro	Fuera del diseño del panel (RF-BIL-07)	E4

3. Estado actual (verificado 2026-10-15, HEAD `680f812`)

3.1 Piezas que sirven y se reutilizan

Pieza	Qué aporta	Veredicto
`lib/whatsapp/meta-client.ts`	`verifyMetaSignature`, `sendWhatsAppText`, `sendWhatsAppAudio`, `getMediaUrl`, `downloadMedia`; cifrado AES-256-GCM por canal (`encryptChannelCredential` / `decryptChannelCredential`, formato `v1:iv:data:tag`, master key `ENCRYPTION_KEY`)	Se conserva; el transporte pasa a Chatwoot
`lib/whatsapp/config.ts`	`TenantWhatsAppConfig` (tono, horario, KB, `preferLocalAi`, `maxTokensResponse`, palabras de derivación, voz) + `mergeWhatsAppConfig` + `isWithinBusinessHours`	Se conserva como **forma del contrato**, pero hoy no se persiste ni se usa en runtime
`lib/whatsapp/resolve-tenant.ts`	`resolveTenantByPhoneNumberId` → `channel_credentials` por `provider + externalId`	Se conserva (ruteo por número dedicado)
`lib/ai/router.ts`	`classifyIntent` (simple/agendamiento/reclamo/otro) y `generateResponse` con fallback local → nube	Se conserva la intención; la política de enrutamiento se corrige (E3.3)
`lib/ai/ollama-client.ts` + `openrouter-client.ts`	Inferencia local (Ollama) y nube (OpenRouter/GLM)	Se conservan; **el prompt es de clínica médica** y hay que reescribirlo por tenant
`lib/voice/{stt,tts}.ts`	`transcribeAudioFromUrl`, `textToSpeech`, `textToSpeechStream`	Se conservan para E3.7
`lib/demos/knowledge.ts`	**La única implementación real de M7**: KB por rubro, `isPriceRequest`, `isInjection`, `applyGuardrails` (post-filtro que borra cifras inventadas)	Se **generaliza** a por-tenant; los demos pasan a ser un consumidor
`lib/entitlements.ts`	`hasFeature` / `setFeature` por canal, con la regla voz ⇒ requiere texto	Se conserva; es la puerta de entrada de los módulos por plan
`lib/db.ts` + RLS + `asAdmin()`	Aislamiento por tenant en una sola conexión, escalando el rol en la transacción	Es la frontera de seguridad del cliente

3.2 Defectos encontrados (los 8, con evidencia)

#	Defecto	Dónde	Qué rompe
1	**Una conversación por mensaje**: la conversación se crea con `id: temp_${tenantId}_${msgId}`	`app/api/webhooks/whatsapp/route.ts:120-130`	RF-MSG-02 (bandeja unificada): no hay hilo
2	**Dos identidades de conversación distintas**: el webhook de n8n usa `contactRef` como id; el de WhatsApp usa `temp_…`	`app/api/webhooks/n8n/route.ts:68` vs `whatsapp/route.ts:120`	El historial del mismo contacto se parte en dos
3	`persistMessage()` es **código muerto** y crea conversaciones con `id: ""`	`whatsapp/route.ts:49-84`	Ruido y una ruta de escritura inválida
4	**La firma del webhook de n8n no puede validar**: lee `req.body` (no existe en App Router, y el body ya se consumió con `await request.json()`), y `timingSafeEqual` tira si las longitudes difieren	`n8n/route.ts:23-45, 61`	Con `N8N_WEBHOOK_SECRET` seteado, **todo** callback de n8n da 401
5	Se escribe `audioId` en `Message`, columna que no existe (la real es `audio_url`)	`whatsapp/route.ts:80` vs `prisma/schema.prisma:149`	El camino de audio falla en runtime
6	`escalationKeywords` hardcodeadas en la ruta, ignorando la config del tenant	`whatsapp/route.ts:15`	RF-KB-04/03: el tono y la derivación no son configurables
7	El prompt del modelo es **de clínica médica** («paciente», «clinica medica», modelo `gemma3:1b`)	`lib/ai/ollama-client.ts:63-70, 107-120`	El producto no es clínico: contamina todos los rubros (FLAG heredado del Plan 8)
8	**No existe KB persistida por tenant**: `knowledgeBase` nunca se lee de la base; el prompt va con string vacío	`lib/whatsapp/config.ts:19` + `mergeWhatsAppConfig` sin consumidores	RF-KB-01/02 incumplibles

4. Decisiones

4.1 Confirmadas (heredadas)

- **D10 (diseño del panel):** el canal es **Chatwoot**, no Meta Cloud API directo. Queda pendiente de este diseño *cómo* se lo integra, que es lo que resuelve E3-D1/D2.
- **Una sola conexión a la base, aislamiento por RLS**, escalando el rol dentro de la transacción (`asAdmin()`). No se reintroduce ningún DSN de administrador.
- **Enrutamiento de números por `channel_credentials`** (`provider: "meta_cloud_api"`, `externalId: phone_number_id`).

4.2 Decisiones del builder (mías, revisables)

ID	Decisión	Alternativa descartada y por qué
E3-D1	**Ingesta por el webhook de cuenta de Chatwoot** (`message_created`) hacia `/api/webhooks/chatwoot`, verificando `X-Chatwoot-Timestamp` + `X-Chatwoot-Signature` = `sha256=` + HMAC-SHA256 de `{timestamp}.{raw_request_body}` con el `secret` del webhook. El webhook directo de Meta (`/api/webhooks/whatsapp`) se retira del camino de producción y queda sólo detrás de un flag para números fuera de Chatwoot.	Mantener Meta directo contradice D10 y obliga a reimplementar la bandeja del operador. Chatwoot ya conecta su inbox de WhatsApp Cloud API contra Meta (RF-MSG-01 se sigue cumpliendo: **proveedor oficial, nunca sesión no oficial**).
E3-D2	**Respuesta por la API de Chatwoot** (`POST /api/v1/accounts/{account_id}/conversations/{conversation_id}/messages`) con el `access_token` del agent bot del tenant. El envío del canal lo hace Chatwoot.	Reenviar por Meta directo partiría el hilo en dos orígenes y perdería los estados de entrega.
E3-D3	**Postgres es el almacén de registro del panel del cliente, del uso y de las cuotas; Chatwoot es la bandeja del operador y el transporte.** Un único módulo escritor espeja cada mensaje a Postgres, idempotente por el id de mensaje de Chatwoot.	Que el panel del cliente lea Chatwoot con un token global mueve la frontera de aislamiento a etiquetas de Chatwoot y rompe la RLS, que es la frontera real de AicoreLink.

ID	Decisión	Alternativa descartada y por qué
E3-D4	**n8n orquesta y agenda; AicoreLink es dueño de la lógica y los datos**, exponiéndolos por `/api/internal/*` con `FACTORY_INTERNAL_API_KEY` (patrón que ya usa el health del panel). Los flujos vivos son JSON versionados en el repositorio.	Poner la lógica de negocio en n8n la saca del control de versiones y de los tests, y contradice «un flujo se clona cambiando sólo parámetros».
E3-D5	**La base de conocimiento son filas estructuradas** (categoría, pregunta, respuesta) y el prompt se arma con presupuesto de caracteres. Sin base vectorial en E3.	Un RAG vectorial agrega infraestructura y un modo de falla nuevo sin resolver la calidad: primero la KB acotada y sus guardarraíles.
E3-D6	**Un número de WhatsApp por tenant** (`channel_credentials` , como hoy). El modo compartido de Aicore (RF-MSG-07) queda fuera de E3.	El ruteo por número dedicado ya funciona y es el que exige la cuenta del cliente; el modo compartido se apoya en la coexistencia (RF-MSX-01), que es E7. **Necesita tu OK (decisión de negocio).**
E3-D7	**La voz (Elite) va en su propia etapa E3.7**, después del núcleo, con su criterio de latencia.	Juntarla con el núcleo mezcla una dependencia de infraestructura (capacidad de la VPS, streaming) con la calidad del texto. **Necesita tu OK (alcance).**

5. Arquitectura

5.1 Camino de un mensaje (objetivo)

```
Cliente final (WhatsApp)
   │ Meta Cloud API  (proveedor oficial)
   ▼
Chatwoot  ── inbox de WhatsApp Cloud API por tenant (una app/credencial por cliente)
   │ webhook de cuenta firmado (message_created)
   ▼
POST /api/webhooks/chatwoot        [AicoreLink]
   ├─ verifica firma (X-Chatwoot-Timestamp + X-Chatwoot-Signature)
   ├─ resuelve el tenant (inbox → tenant, vía channel_credentials.externalId)
   ├─ espeja conversación + mensaje en Postgres (idempotente, RLS)
   ├─ suma uso (TenantUsage) y evalúa cuota (80% / 100%)
   ├─ si el hilo está derivado o la cuota se agotó → no responde; avisa (RF-MSG-03/§3.2)
   ├─ guardarraíles de entrada (prohibidos, inyección, precio) → respuesta canónica
   ├─ enruta el modelo (local simple / nube complejo) y genera con la KB del tenant
   ├─ guardarraíles de salida (cifras inventadas, sin dato → «no tengo ese dato» + derivar)
   └─ responde por la API de Chatwoot  →  Chatwoot  →  Meta  →  cliente
```

5.2 Capas de IA (RF-KB-02/03/05)

1. **Entrada:** temas prohibidos y patrones de inyección nunca llegan al modelo (regla, respuesta canónica). Si no hay dato en la KB, se responde «no tengo ese dato» + se ofrece derivar, sin llamar al modelo.
2. **Prompt:** instrucciones del sistema reconstruidas por tenant y rubro (español de Chile, tuteo), KB acotada, tono, horario.
3. **Salida:** post-filtro que borra cualquier cifra monetaria ausente de la KB (generalización de `applyGuardrails`), y derivación si el texto queda vacío.
4. **Derivación:** por intención (`reclamo`), por palabra clave configurable, por tema prohibido o por pedido explícito. El hilo queda derivado, con resumen, y la IA calla hasta que se reactive.

5.3 Uso, cuotas y costo (SRD §3.2 / RC-04 / RF-KB-07)

- `TenantUsage` acumula por `(tenant, canal, período)`: mensajes entrantes, salientes, tokens y costo estimado (con `CostRate`).
- El límite del plan sale de `lib/panel/plans.ts` (§3.2: 300 / 1.500 / 5.000 conversaciones IA; plantillas 0/300/1.000; cambios 1/3/8; voz —/—/120 min).
- Al 80% y al 100% se avisa al cliente y al operador. Al agotarse, **el asistente no se apaga en silencio**: deriva a una persona.
- Cada respuesta registra el modelo usado y los tokens ⇒ el costo por conversación es auditable (RF-KB-07, RF-QLT-06).

---

6. Lo que queda para vos (Leonardo)

#	Decisión	Mi recomendación
1	**Numeración (RF-MSG-07)**: ¿un número dedicado por tenant, o el modo compartido de Aicore para Starter?	Número dedicado en E3; el compartido se apoya en la coexistencia (E7) y hoy no hay ningún tenant Starter con número real
2	**Voz (Elite)**: ¿E3.7 al final de esta etapa, o se va con E7?	Al final de E3, en su propia etapa, para que el núcleo de texto no dependa de la latencia
3	**Chatwoot como transporte**: ¿lo confirmás, sabiendo que el webhook de Meta directo sale del camino de producción?	Confirmado (ya es D10); el número propio de AicoreLink quedaría igual dentro de Chatwoot

Si las tres te cierran, el plan atómico (`…-E3-mensajeria-ia-plan.md`) sale con estas decisiones ya fijadas.

---

7. Etapas y criterios de cierre

Cada etapa cierra con los gates del proyecto: `lint` , `build` , `typecheck` , `vitest` , y el E2E local que ya existe.

Etapa	Contenido	Cierra cuando
E3.1 Ingesta e identidad de conversación	Firma del webhook de Chatwoot, resolver tenant por inbox, espejo idempotente en Postgres, una sola identidad de conversación, retiro del camino Meta directo, los 8 defectos de §3.2 corregidos	Dos mensajes del mismo contacto caen en **una** conversación, en orden; la firma inválida se rechaza (test); no queda código muerto
E3.2 KB y guardarraíles por tenant	KB estructurada, temas prohibidos, tono, prompt por rubro, guardarraíles de entrada/salida, generalización de `lib/demos`	50 preguntas fuera de base ⇒ ≥ 95% «no tengo ese dato» + derivación; las pruebas adversarias de inyección no pasan ninguna regla; los tests del chat público siguen verdes
E3.3 Enrutamiento, uso y cuotas	Política de modelo por plan, registro de tokens/costo por conversación, `TenantUsage` , avisos 80%/100%, derivación al agotar	El uso coincide con los registros de mensajes; los avisos disparan en 80% y 100%; el asistente deriva en vez de callarse
E3.4 Derivación con resumen + modo aprendizaje	Estados del hilo, resumen al derivar, IA en pausa hasta reactivar, aviso al dueño, pregunta sin respuesta → respuesta aprobada → KB	Derivar por palabra clave e intención marca el hilo, avisa al dueño y la IA calla; una respuesta aprobada se usa desde la consulta siguiente
E3.5 Flujos n8n versionados	Los 6 flujos en JSON en el repo (`lead` , `respuesta` , `reserva` , `recordatorio` , `reseña` , `reporte`), parámetros por tenant, importación documentada, llamadas a `/api/internal/*`	Clonar un flujo para un tenant nuevo es sólo parámetros; el flujo dispara y vuelve con firma válida
E3.6 Calidad y transparencia	Registro de modelo/versión por respuesta, tiempo a primera respuesta, evaluación por muestreo, suite adversaria	El medidor de primera respuesta se calcula con datos reales por tenant; el modelo y su versión quedan registrados; la suite adversaria corre en los gates
E3.7 Voz (Elite) — *si va*	Recepcionista de voz con aviso de IA y de grabación, ≤ 1,5 s por turno	Una llamada de prueba agenda una hora sin intervención humana

---

8. Riesgos

Riesgo	Impacto	Mitigación
No existe todavía una instancia de Chatwoot ni un número de Meta con credenciales	La verificación en vivo queda bloqueada; el código se construye y se prueba con dobles	E3.1 arranca con la firma y el espejo probados por tests de integración; la verificación en vivo se declara como pendiente de infraestructura, nunca como «listo»

Riesgo	Impacto	Mitigación
Cambiar `lib/demos` para compartir los guardarraíles puede romper el chat público de la landing	Regresión visible en el embudo	Los tests del chat público son parte del gate de E3.2; la generalización es un refactor con la suite existente en verde
El modelo local hoy es `gemma3:1b` y el estándar del proyecto es Mistral	Calidad por debajo del objetivo del 90%	E3.2 fija el modelo por configuración y E3.6 lo mide; el prompt clínico se reescribe en E3.2
La cuota del plan no está implementada en ningún lado	Un cliente puede agotar su plan sin aviso	E3.3 antes de abrir el canal a un cliente real
Retirar el webhook de Meta directo	Si un tenant tiene número fuera de Chatwoot, deja de recibir	Se conserva detrás de un flag y el ruteo se decide por `channel_credentials`; se documenta la excepción

9. No-objetivos

- No se toca `consultorio-medico`.
- No se comparten base, credenciales ni flujos con AicoreMed.
- No hay portal de usuario final ni pasarela de pago.
- No se rediseña la landing ni el panel (eso es F2–F7).
- No se implementa RAG vectorial, ni fine-tuning, ni entrenamiento de modelos.